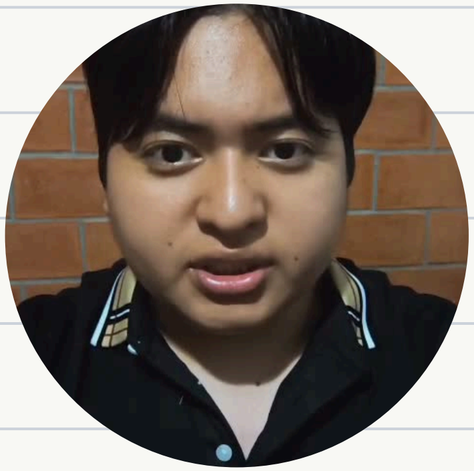
Industrias Robloxianas



Axel Javier Rosas Rodríguez | A01738607
Elian Cantalapiedra Sabugal | A01738462
Edwin Emmanuel Salazar Meza | A01738380

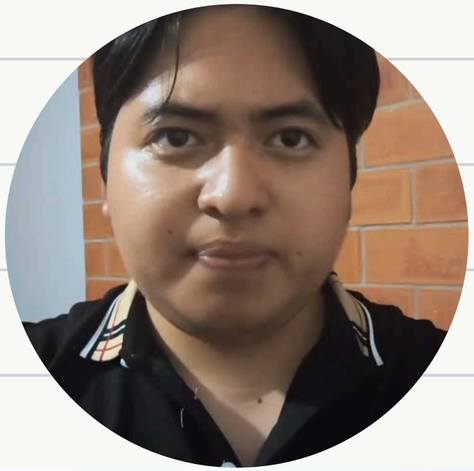
Introduction

Problem to be Solved: Configure a differential robot to navigate point-to-point autonomously while detecting and dynamically reacting to a simulated traffic light.

Solution Strategy

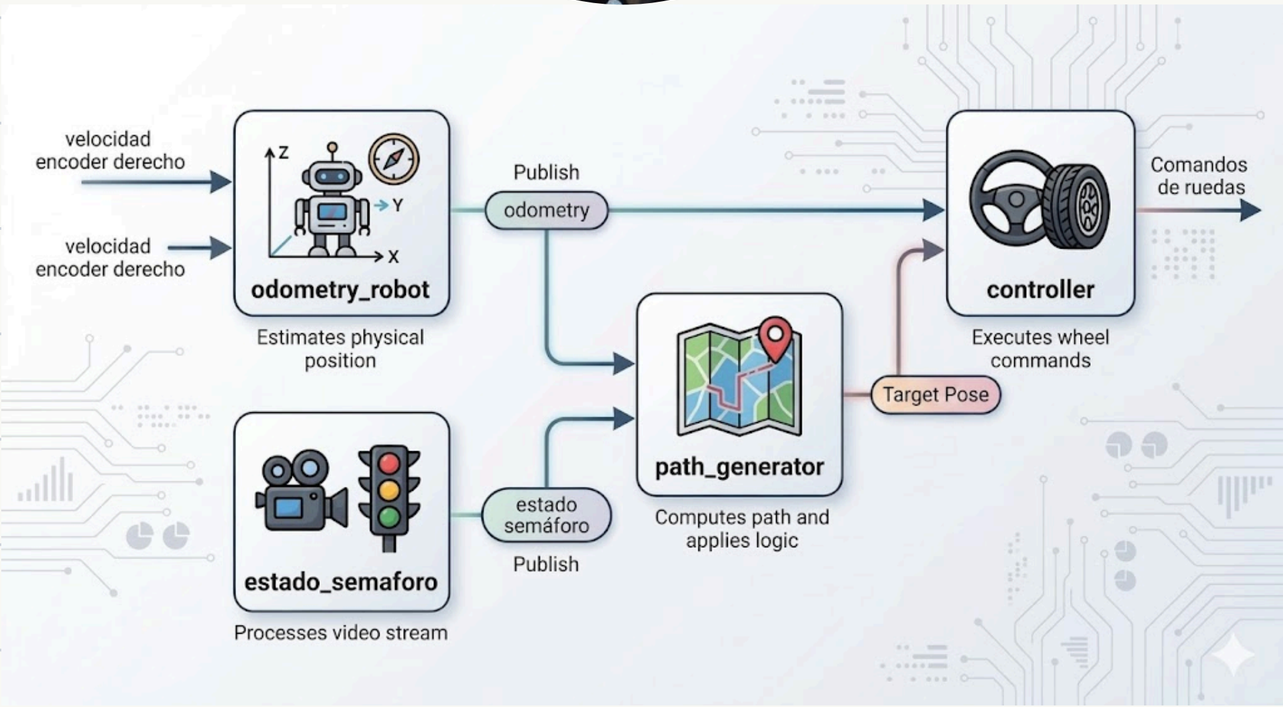
- **Perception:** Computer vision using HSV color space segmentation.
- **Navigation:** State machine for trajectory and goal generation.
- **Control:** Closed-loop PI controller to manage kinematics and physical disturbances.

Project Structure

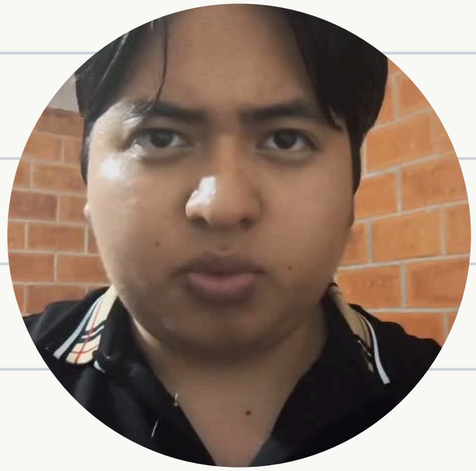


Node Architecture:

- **odometry_robot:** Estimates physical position.
- **estado_semaforo:** Processes video stream.
- **path_generator:** Computes path and applies logic.
- **controller:** Executes wheel commands.



Controller



Advantage:

- The integral term K_i successfully eliminates steady-state error, absorbing the physical disturbances caused by slipping tires.

Disadvantage:

- Environmental Dependency: The current gains are over-fitted for smooth surfaces. Operating on a rough, high-friction surface would cause aggressive overshooting and jerky movements due to excessive control effort.

```python
def control_linear(self, v_ref):
    e = v_ref - self.v_real
    self.e_v_int += e * self.dt
    self.e_v_int = np.clip(self.e_v_int, -0.5, 0.5) # Anti-windup
    u = self.Kp_v * e + self.Ki_v * self.e_v_int
    return float(np.clip(u, -0.4, 0.4))

def control_angular(self, w_ref):
    e = w_ref - self.w_real
    self.e_w_int += e * self.dt
    self.e_w_int = np.clip(self.e_w_int, -0.5, 0.5) # Anti-windup
    u = self.Kp_w * e + self.Ki_w * self.e_w_int
    return float(np.clip(u, -4.0, 4.0))
```

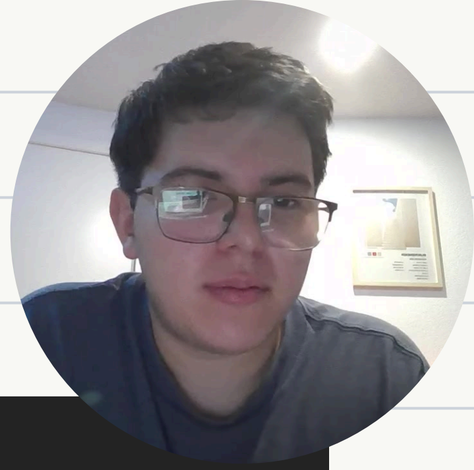
Traffic light status

```
# Candado para asegurar que solo se procese un frame a la vez
self.lock = Lock()

# Nombre de ventana fijo y creación única
self.window_name = "Deteccion_Semaforo_ROI"
cv2.namedWindow(self.window_name, cv2.WINDOW_AUTOSIZE)

self.config_hsv = {
    'rojo': [((0, 100, 100), (10, 255, 255)),
             ((160, 100, 100), (180, 255, 255))],
    'amarillo': [((10, 100, 100), (45, 255, 255))],
    'verde': [((46, 100, 100), (80, 255, 255))]
}

self.p_ancho = 0.45
self.p_alto = 0.95
self.MIN_AREA = 170
self.MAX_AREA = 1000
```

```
def obtener_roi(self, frame):
    alto, ancho = frame.shape[:2]
    centro_x, centro_y = ancho // 2, alto // 2
    ancho_roi, alto_roi = int(ancho * self.p_ancho), int(alto * self.p_alto)

    x1, y1 = max(0, centro_x - (ancho_roi // 2)), max(0, centro_y - (alto_roi // 2))
    x2, y2 = min(ancho, centro_x + (ancho_roi // 2)), min(alto, centro_y + (alto_roi // 2))

    roi = frame[y1:y2, x1:x2]
    frame_visual = frame.copy()
    cv2.rectangle(frame_visual, (x1, y1), (x2, y2), (0, 255, 0), 2)

    return roi, frame_visual, (x1, y1)
```

- Processes the camera video in real time.
- Detects the traffic light color and publishes the state for the control logic.
- Uses a Lock to process only one frame at a time.
- If a new image arrives while another is being processed, it is discarded.
- Extraction of a centrally located Region of Interest (ROI) defined empirically.
- Conversion from RGB to HSV.
- Greater robustness against changes in ambient lighting.

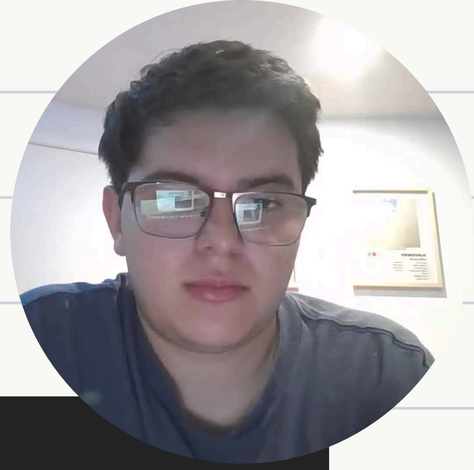
Traffic light status



- Segmentation and filtering, generation of binary masks using inRange.
- Contour extraction to eliminate visual noise.
- Final validation: selection of the contour with the largest area.
- Verification using minimum and maximum thresholds.
- Color confirmation only if it meets the defined criteria.

```
for color, rangos in self.config_hsv.items():
    mask_final = np.zeros(hsv_roi.shape[:2], dtype=np.uint8)
    for lower, upper in rangos:
        mask = cv2.inRange(hsv_roi, np.array(lower), np.array(upper))
        mask_final = cv2.add(mask_final, mask)

    contorno = self.obtener_contorno_valido(mask_final)
    if contorno is not None:
        estado = color.upper()
        c_detectado = contorno
        break

if c_detectado is not None:
    (x, y), radio = cv2.minEnclosingCircle(c_detectado)
    center = (int(x + x1), int(y + y1))
    radius = int(radio)
    cv2.circle(frame_visual, center, radius, (0, 255, 255), 2)
```

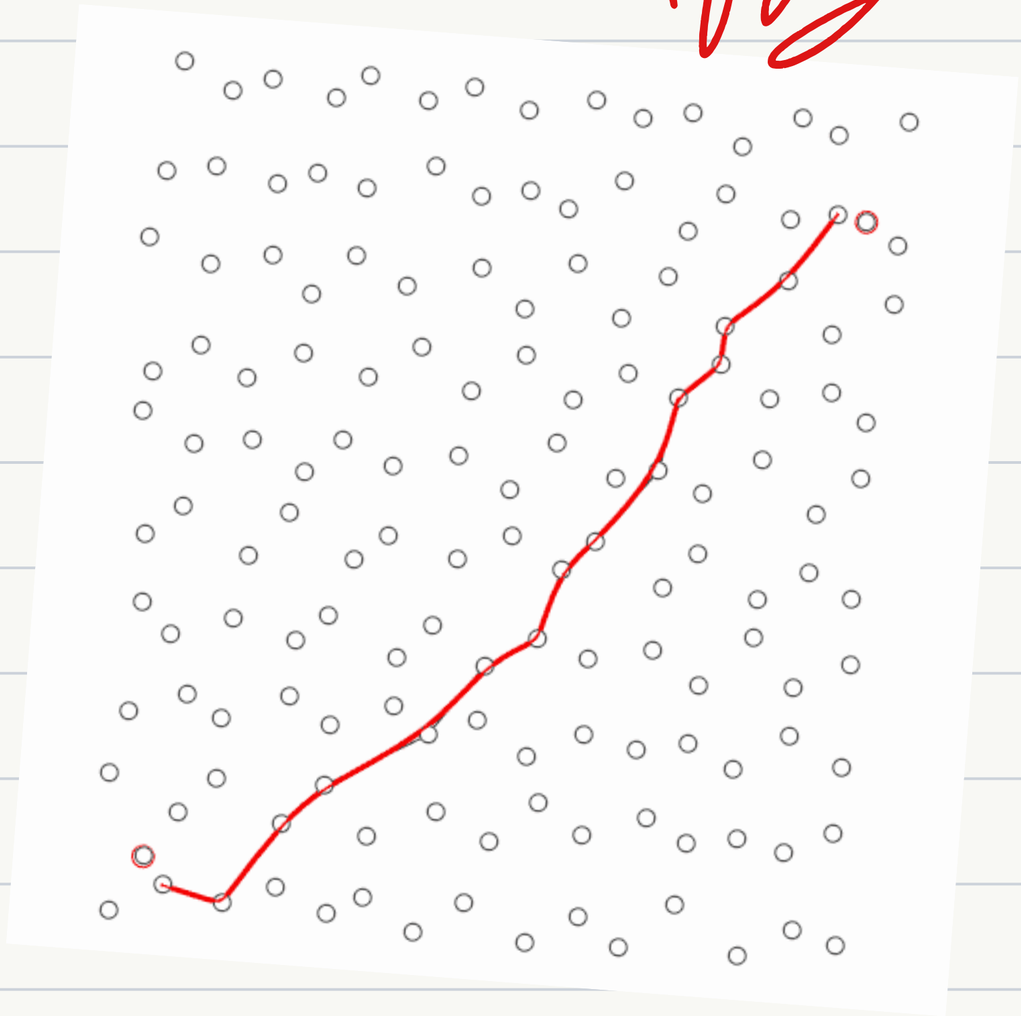
Traffic light status



- Segmentation and filtering, generation of binary masks using inRange.
- Contour extraction to eliminate visual noise.
- Final validation: selection of the contour with the largest area.
- Verification using minimum and maximum thresholds.
- Color confirmation only if it meets the defined criteria.

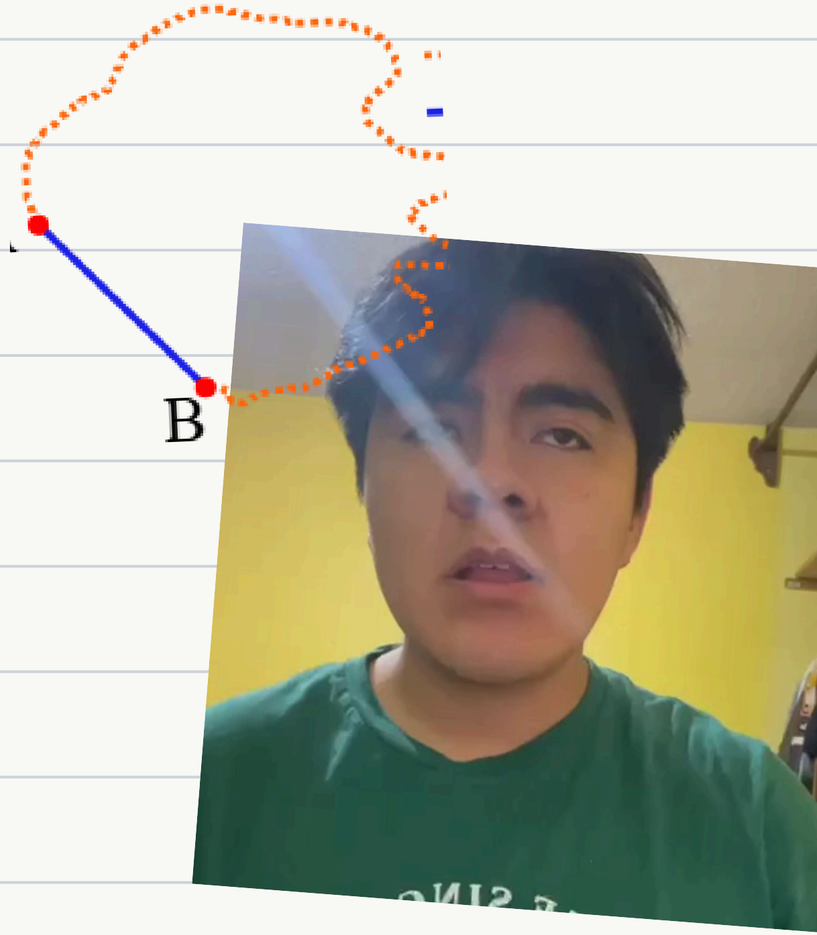
```
for color, rangos in self.config_hsv.items():
    mask_final = np.zeros(hsv_roi.shape[:2], dtype=np.uint8)
    for lower, upper in rangos:
        mask = cv2.inRange(hsv_roi, np.array(lower), np.array(upper))
        mask_final = cv2.add(mask_final, mask)

    contorno = self.obtener_contorno_valido(mask_final)
    if contorno is not None:
        estado = color.upper()
        c_detectado = contorno
        break

if c_detectado is not None:
    (x, y), radio = cv2.minEnclosingCircle(c_detectado)
    center = (int(x + x1), int(y + y1))
    radius = int(radio)
    cv2.circle(frame_visual, center, radius, (0, 255, 255), 2)
```

Path Generator



Goal and Error Management

- **Target List:** It stores a sequence of (x, y) coordinates that the robot must visit sequentially.
- **Distance Calculation:** It continuously measures the Euclidean distance between the current position (estimated by odometry) and the target point.
- **Orientation Error:** It calculates the exact angle the robot should be pointing at target_angle and compares it with its current orientation to obtain the error_theta.
- **Tolerance:** If the robot enters a radius of 5 to 10 cm from the goal, it assumes it has reached it and automatically jumps to the next point in the list.

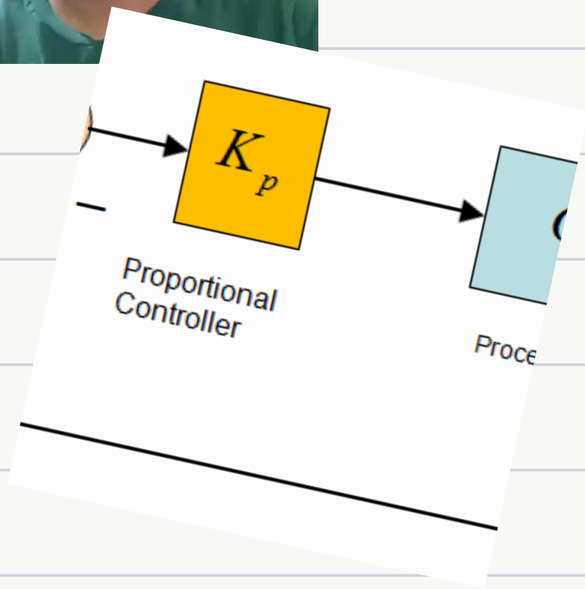


Velocity and Safety Logic

- **Proportional Control:** It generates a linear velocity proportional to the distance and an angular velocity proportional to the orientation error, using constants k_linear and k_angular.

- **Turning Prioritization**: If the orientation error is greater than ~30° (0.52 radians), the node nullifies the linear velocity and allows the robot only to rotate on its own axis. This ensures the robot is correctly aimed before advancing, avoiding erratic deviations.

- **Physical Limits:** It applies saturation so that the robot never exceeds hardware constraints (0.4m/s and 4.0 rad/s).

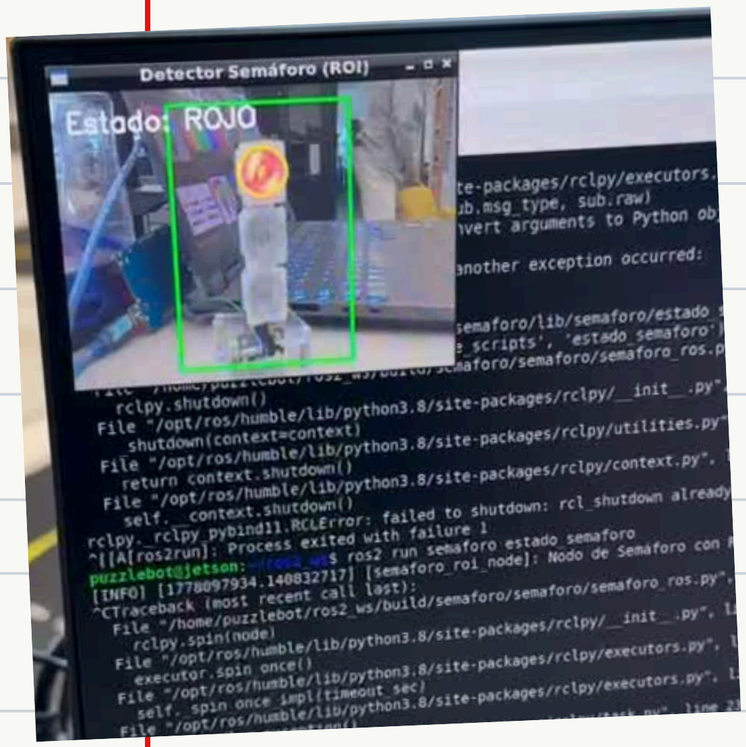


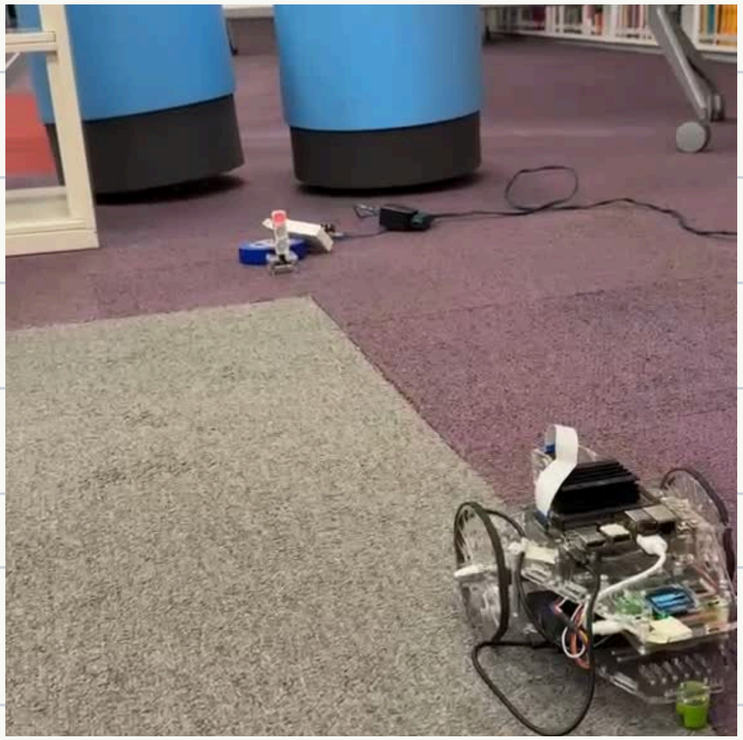
Traffic Light Modulation

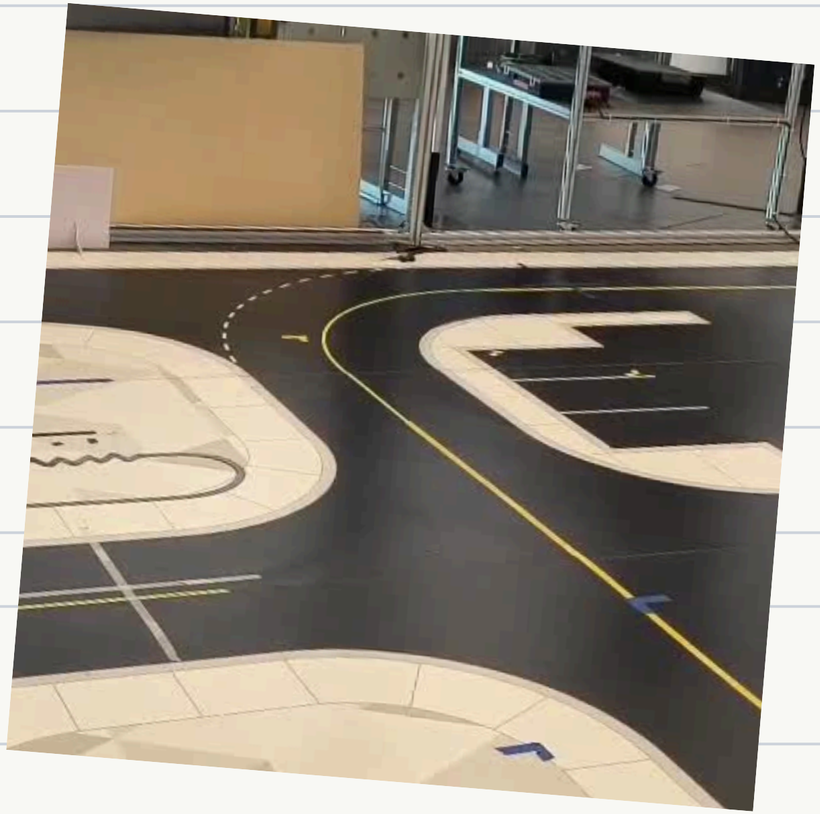
This is the decision-making layer that alters nominal performance based on visual perception:

- **Green:** Multiplies velocities by 1.0, allowing normal advancement.
- **Yellow:** Multiplies by a factor of 0.2, forcing a slow and cautious drive.
- **Red:** Multiplies by 0.0, canceling signals to the motors to stop and wait until the light changes

Evidence


Detector Semáforo (ROI)
Estado: ROJO
rclpy.shutdown()
File "/opt/ros/humble/lib/python3.8/site-packages/rclpy/__init__.py"
shutdown(context=context)
File "/opt/ros/humble/lib/python3.8/site-packages/rclpy/utilities.py"
return context.shutdown()
File "/opt/ros/humble/lib/python3.8/site-packages/rclpy/context.py"
self._context.shutdown()
rclpy._rclpy_pybind11.RCLError: failed to shutdown: rcl_shutdown already
^[[A[ros2run]: Process exited with failure 1
puzzlebot@jetson:
$ ros2 run semaforo estado_semaforo
[INFO] [1778097934.140832717] [semaforo_roi_node]: Nodo de Semáforo con
^CTraceback (most recent call last):
File "/home/puzzlebot/ros2_ws/build/semaforo/semaforo/semaforo_ros.py"
rclpy.spin(node)
File "/opt/ros/humble/lib/python3.8/site-packages/rclpy/__init__.py"
executor.spin_once()
File "/opt/ros/humble/lib/python3.8/site-packages/rclpy/executors.py"
self._spin_once_impl(timeout_sec)
File "/opt/ros/humble/lib/python3.8/site-packages/rclpy/executors.py"





Conclusion

The system successfully validated the integration of probabilistic perception and deterministic control. From a theoretical perspective, the use of a closed-loop PI controller demonstrated that steady-state error and disturbances can be effectively mitigated through precise gain tuning.

Practically, the implementation revealed that while HSV color segmentation is effective for controlled environments, its robustness is limited by ambient lighting and scene complexity. Furthermore, the mechanical performance remains highly dependent on the friction coefficient of the surface, indicating that for universal deployment, the control algorithms must transition from fixed gains to adaptive or non-linear models to maintain stability across varying terrains.